

3/27/2026

Score Breakers

Programmering B eksamen

Synopsis lavet af: Cornelius Struer Lindemann

Program lavet af: Cornelius Struer Lindemann og Emil Toftbo Van Dijk

Repository: https://github.com/B6rnard/Score_breakers

Contents

Opgaveformulering	2
Projektbeskrivelse	3
Kravspecifikation.....	3
Need to have/Nice to have	4
Gruppefordeling	4
Brug af AI	4
Funktionsbeskrivelse	5
Spil.....	5
Leaderboards	8
Flask server	9
Profiler.....	10
Dokumentation	11
Kode eksempler	12
Test	14
Konklusion	14
Bibliografi	15
Bilag.....	15

Opgaveformulering

Tema 1. Website med eget tema

I skal udvikle et program eller en applikation som demonstrerer brug struktureret brug af trelagsmodellen. Det vil sige at løsningen skal indeholde et data-, et logik- og et præsentationslag. Datagrundlaget og præsentationslaget er valgfrit - I bestemmer altså selv hvad løsningen handler om og hvordan den præsenteres for brugeren.

Projektbeskrivelse

Vores produkt skal være en applikation med spil hvor at du kan sætte dine egne highscores, konkurrerer med dine venner og se et live leaderboard over dine og dine venners highscores. Vi kommer til at have nogle unikke spil inde på vores applikation som du kan spille hvor at dine rekorder bliver direkte gemt, så du kan gå ind og se dem. Man kommer også til at have mulighed for at lave et "party" som vil åbne en flask server hvor at dine venner på det samme internet kan se dine highscores og selv prøve at slå dem.

Kravspecifikation

1. Brugeren skal kunne oprette en konto og logge ind:

Vi kunne godt tænke os at lave en profil side hvor man både kan lave profiler og logge ind på allerede eksisterende profiler. Vi vil også gerne have at navnet på profillen der har sat rekorden står ved siden af så det er nemmere at se hvem der har spillet og lavet hvilke rekorder.

2. Brugeren skal kunne vælge mellem minimum tre spil.

Siden vores program er en "game hub" skal man have nogle forskellige spil man kan lave rekorder i for at give noget variation og en bedre bruger venlighed. Vores ide med "game hub" er inspireret af fx. Y8, Friv og Steam.

3. Spillene skal have en fungerende spilmechanik (bevægelse, point, game over).

Selfølgelig siden at man skal optimalt kunne spille og sætte rekorder skal spillene ikke bare være en prototype men faktisk noget der er muligt at spille. Det er også svært at få den samlede ide af vores applikation hvis spillene ikke er fungerende.

4. Systemet skal registrere og gemme brugerens highscore.

Dette krav er meget vigtigt siden at hele ideen ved vores applikation er at man skal kunne konkurrerer med sine venner hvilket ikke er ligeså fedt at gøre hvis applikaitonen ikke automatisk gemmer highscores of profiler.

5. Brugeren skal kunne se sin egen highscore på sin profil.

Dette krav er ikke ligeså vigtigt at have siden man kan tjekke leaderboards og se hvor ens eget navn ligger, men det er mere brugervenlighed så du kan se det nogle forskellige steder og give effekten af en mere fuldent profil hvis du sætter mange rekorder.

Need to have/Nice to have

Brugeren skal kunne oprette en konto og logge ind:	Need to have
Brugeren skal kunne vælge mellem minimum tre spil.	Nice to have
Spillene skal have en fungerende spilmekanik (bevægelse, point, game over).	Need to have
Systemet skal registrere og gemme brugerens highscore.	Need to have
Brugeren skal kunne se sin egen highscore på sin profil.	Nice to have

Gruppefordeling

I dette afsnit ønsker vi at gå over hvilke roller de forskellige gruppemedlemmer er blevet tildelt og hvad vi hver især fokusere på.

Cornelius fokuserer primært på GUI og vil have fokus på at alle interaktionselementer virker effektive og ser behageligt ud og følger temaet.

Emil fokuserer primært på Logik laget og vil derfor sørge for at der er en effektiv og forhåbentligt nem oversættelse fra data laget til præsentationslaget.

Brug af AI

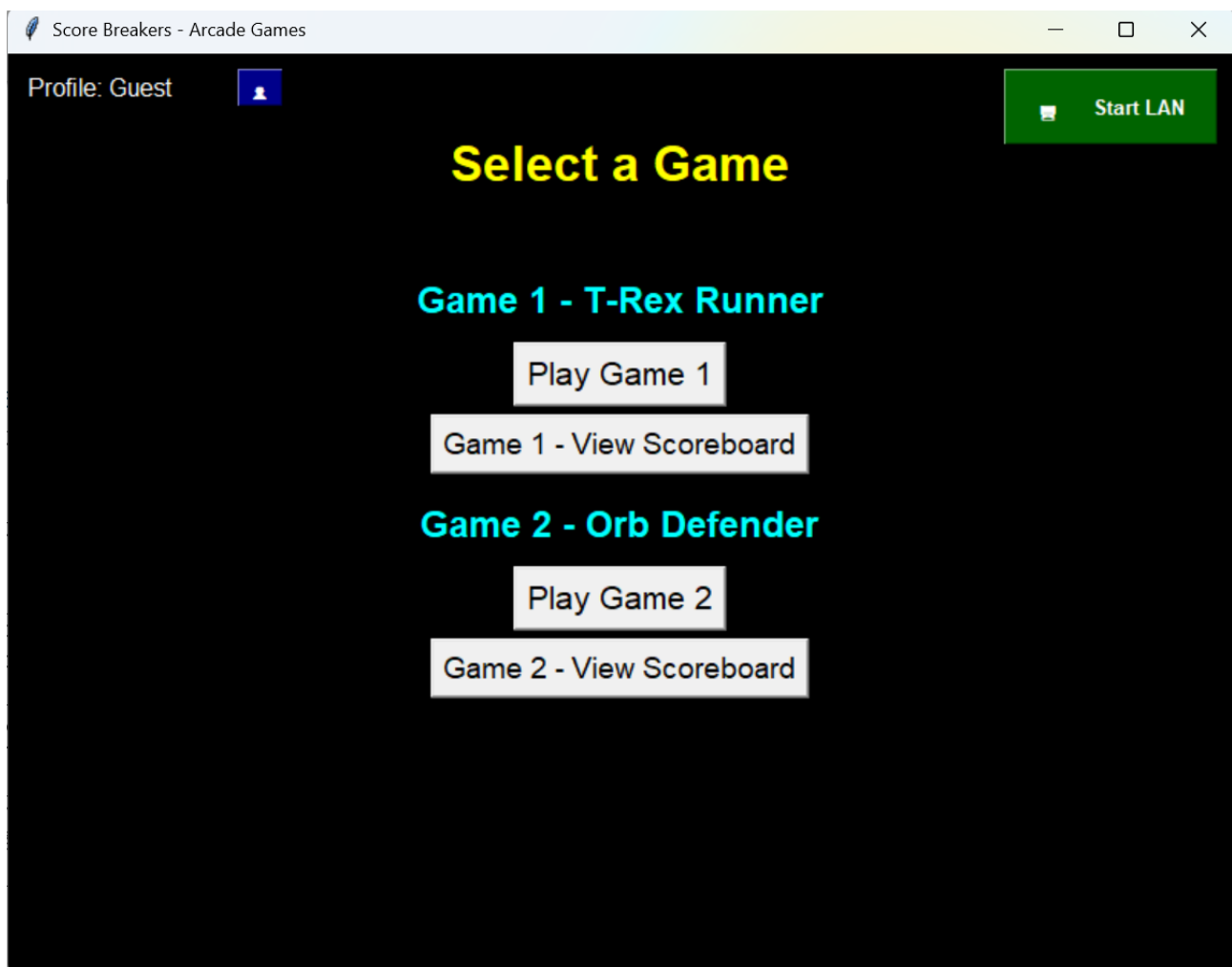
Vi har taget brug af AI specifikt den indbyggede AI i visual studio code, GitHub CoPilot. Vi har også fået alle vores beskeder både til og fra AI dokumenteret, og vi har primært brugt AI til både at optimere kode processen og sprog som vi ikke har så meget forstand på som HTML og hvordan vi gemmer highscores i en json fil. Som sagt er alt AI dokumenteret.

Funktionsbeskrivelse

Vores program har en masse funktioner og nogle som kan være lidt komplicerede hvilket vi vil gå over i dette afsnit, vi har også en README. Fil for at hjælpe folk med at spille med deres venner og lave en LAN server.

Spil

Alle vores spil er tilgængelige på forsiden af vores applikation som du automatisk starter i hver gang du åbner applikationen. Vi har valgt at tage stor inspiration af allerede eksisterende populære arkade spil, men ændret designet og spil mekanikken for ikke at direkte kopiere.



Figur 1 - menu

Vi har lige nu 2 forskellige tilgængelige spil som man kan spille og få nye highscores som automatisk bliver registreret i leaderboards. For at spille spillene skal man trykke på knappen der hedder "Play Game" under det spil du ønsker at spille.

Det første spil vi har lavet er T-rex runner hvilket er inspireret af det originale T-rex runner som kommer op når man ikke har noget internet på Google.



Figur 2 - trex runner

I dette spil skal man hoppe over nogle forhindringer og din score er baseret på hvor langt tid du har været i live og ikke på hvor mange forhindringer du har hoppet over, til forskel for vores andet spil. Hastigheden du bevæger dig på ændre sig ikke hen over tiden det er bare hvor langt tid man kan holde sig i live.

I både dette spil og vores andet har jeg fået arbejdet af GUI og vi har valgt at holde det meget simpelt så det faktisk kunne ligne noget fra en arkade samtidig med at det er funktionelt, men jeg har lagt meget vægt på at det skal ligne et gammelt arkade spil.



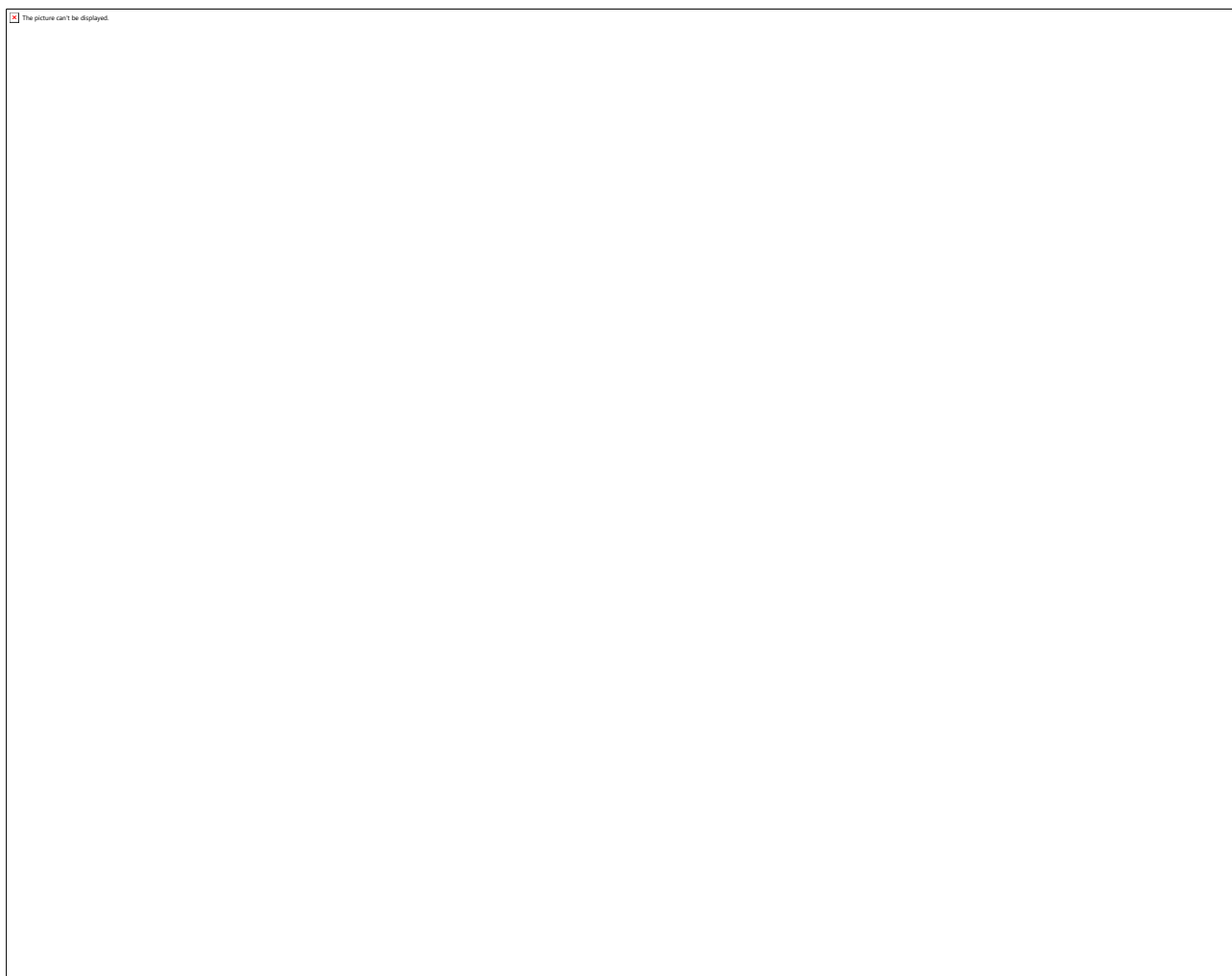
Figur 3 - orb defender

Dette som vist på billedet er vores andet spil som er inspireret af klasikeren "Space Evaders", i dette spil falder der nogle kugler ned fra toppen af skærmen hvor at man som en kanon skal skyde dem inden de rammer gulvet. Man bruger pil tasterne for at gå til siden og man skyder med mellemrum knappen, i dette spil der får du point for hver kugle du skyder i stedet for hvor langt tid du er i live.

Ligesom det andet spil har jeg haft fokus på at det skulle spille godt ind i vores gamle arkade stil, det er også en af grundene til at vi har valgt at tage inspiration fra "Space Evaders".

Leaderboards

Vores leaderboards er også ligesom spillende tilgængelige på forsiden af applikationen hvor at under "Play Game" knappen er der en "View Scoreboard" knap under hvert spil.



Figur 4 - leaderboard

Igen har vi gået med et meget simpelt design på leaderboardet for at nemmere have en GUI der ligner noget fra en arkade. På leaderboard skærmen kan man se i rækkefølge efter højest, hvilke scores folk har lavet og hvem der har lavet dem så man nemt kan finde rundt i sine egne og sine venners på en lokal computer.

Flask server

Hvis man ikke spiller på den samme computer er der vores LAN server hvor at man på et lokalt internet kan åbne en LAN server for at se hindanens rekorder.



Figur 5 - flask

Når man laver en LAN server, så ved hjælp af Flask åbner du en lokal hjemmeside som både du og dine venner kan gå ind på for at se alt data i .json filerne, hjemmeside koden til flask serveren bliver og vist oppe i højre hjørne af applikationen når man trykke på den grønne start LAN knap. Det er dog meget vigtigt at man er på det samme internet, ellers er det ikke muligt at tilslutte sig til Flask serveren.

Profiler

Vores profil knap ligger helt oppe i venstre hjørne hvilket er markeret med ansigtsløs person for at spille ind i vores arkade stil.



Figur 6 - profiles

Når man trykker på profil knappen kommer man ind på denne her tab hvor at man har alle profil funktionerne. Du kan se hvilken profil du er på lige nu, skifte profiler til tidligere oprettede og lave nye profiler. Man behøver ikke et kodeord til sin profil og vi ser det ikke nødvendigt siden at folk ikke kan slette rekorder eller profillen aligevel. Alle de funktioner hjælper os med at opfylde vores krav om profiler.

Dokumentation

I dette projekt har vi valgt at strukturere vores applikation efter trelagsmodellen, hvilket betyder at programmet er opdelt i et datalag, logiklag og GUI lag. Denne opdeling har gjort det nemmere for os at udvikle og teste vores program siden vi også har delt ansvaret.

Data laget var det vi havde delt ansvar imellem, så det var ikke en af vores fokuspunkter men det bestod primært af hvordan man håndterede JSON filer hvor at vores profiler og rekorder blev gemt. Vi valgte JSON fordi at det er nemt at arbejde med i Python og hjælper med at give en mere overskuelig struktur. Funktionerne i datalaget sørger også for at gemme nye rekorder, se allerede eksisterende data og at opdatere profiler og rekorder.

Logik laget fungere som sådan broen mellem data og brugergrænseoverfladen, det håndtere spilmekanikken som point beregning, når spillet slutter og opdatering af rekorder. Det er også logik laget der styrer hvordan at dataen bliver behandlet inden den bliver vist i GUI laget.

GUI laget består af både spilgrænseoverfladen og menuerne, her har vi arbejdet med at lave et simpelt og overskueligt design inspireret af klassiske arkadespil. GUI laget håndtere bruger input og sender det videre til logiklaget.

Vi har også implementeret en Flask server som gør det muligt at dele highscores over det samme internet.

Programmet starter i en main.py fil som samler de forskellige dele af vores kode og sørger for at de arbejder sammen, for eksempel at implementere vores spil ind i TKInter applikationen. Samlet set fungere programmet ved at brugeren interagerer med GUI laget, hvor at logik laget behandler inputtet og enten opdatere dataen eller laver et return med et resultat.

*Figur 7 – flowchart*

Jeg har også lavet et flowchart på Miro som forklare hvordan vores applikation er bygget op hvis den virker uoverskuelig at navigere i.

Kode eksempler

Her har jeg tænkt mig at implementere få kode eksempler for at illustrere hvordan programmet virker:

```
def add_score(self, game_key: str, player: str, score: int) -> None:
    """
    Add a score entry for a game.

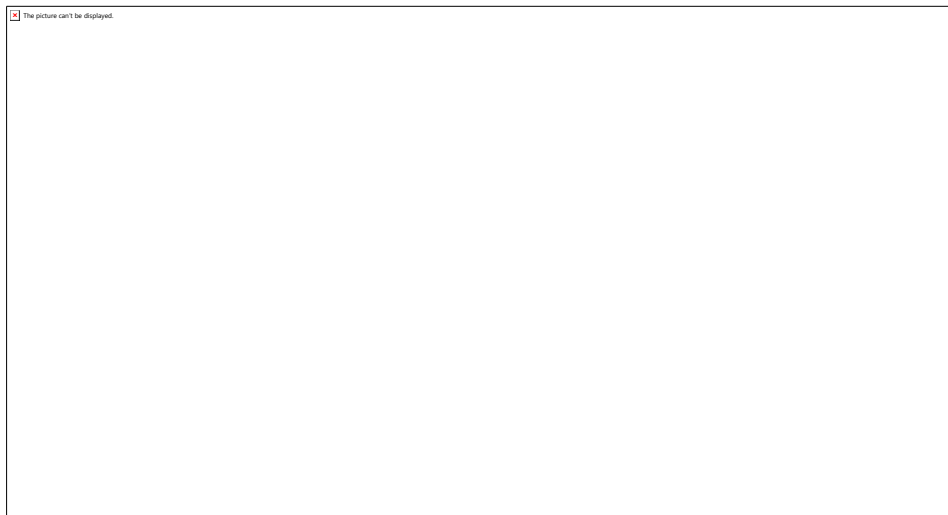
    - game_key: e.g. "game1", "game2"
    - player: profile name
    - score: numeric score
    """
    if game_key not in self._data:
        self._data[game_key] = []

    self._data[game_key].append({"player": player, "score": int(score)})

    # Sort descending by score (highest first)
    self._data[game_key].sort(key=lambda s: s.get("score", 0), reverse=True)
```

Figur 8 - gem til JSON

I dette billed jeg har medtaget kan man se hvordan at programmet opdatere rekorder med brugernavn, hvilket spil og hvad rekorden var som set i def funktionen. Derefter kan vi også se nede i bunden af billedet hvordan at programmet sortere de rekorder der bliver sat i JSON filen.



Figur 9 - load scores til Flask

I dette billed har jeg medtaget koden der henter vores data og rekorder fra JSON filen og viser dem på vores Flask server. Hvilket er muligt at gøre så nemt på grund af at vores JSON fil er lavet som et dictionary.

```
1  {
2  "game1": [
3    {
4      "player": "Corner",
5      "score": 650
6    },
7    {
8      "player": "Emil",
9      "score": 428
10   },
11   {
12     "player": "Corner",
13     "score": 209
14   },
15   {
16     "player": "Guest",
17     "score": 96
18   }
19 ],
```

Figur 10 - Dictionary eksempel

Her kan man se hvordan at koden er skrevet i vores JSON fil, hvordan programmet sortere highscores, brugernavne og spillene i et dictionary.

Test

For at sikre at vores program fungerer som forventet, har vi arbejdet med test af de vigtigste funktioner. Vi har primært fokuseret på at teste logiklaget og datalaget, da disse er de mest kritiske for programmets funktionalitet.

Vi har anvendt test til at teste funktioner som:

- Gemning af highscores
- Hentning af data fra JSON
- Opdatering af eksisterende profiler
- Korrekt sortering af leaderboards

Vores tests er opbygget efter simpelt med (AAA) eller arrange, act og assert for at sikre at vi laver en god test (Arrange-Act-Assert: A Pattern for Writing Good Tests, n.d.). først opstiller vi testdata, så gør vi den handling vi vil teste for også ser vi om resultatet er som vi gerne vil have.

Et eksempel kunne være at teste hvor vi indsætter en ny highscore, derefter tjekker om den bliver gemt korrekt og vises på leaderboardet i den rigtige rækkefølge.

Det er lidt sværere at teste i forhold til vores GUI, siden det er svært at automatisk at teste brugerinteraktioner. Vi har dog anvendt en manuel test hvor at vi gennemgår programmets funktioner for at sikre os at alt virker som det skal og der ikke er nogle fejl.

Samlet set viser vores test at vores program virkede meget stabilt og uden fejl vi kunne finde i de vigtige funktioner. Vi har også snakket med nogle fra vores klasse (Stefan og Jeppe) angående det design vi prøvede at gå efter hvor vi fik af vide, at vi var gode til at ramme den æstetik vi gik efter med gammel klassisk arkade.

Konklusion

I dette projekt har vi udviklet en applikation i form af en game hub hvor brugere kan spille forskellige spil, sætte highscores og konkurrere med hinanden.

Formålet med projektet var at lave et program som demonstrerer brugen af trelagsmodellen, og dette mener vi at vi er lykkedes med. Opdelingen i data laget, logik laget og GUI laget har gjort det nemmere at strukturere vores kode og skabe overblik over hvordan de forskellige dele af programmet hænger sammen.

Vi har opfyldt de vigtigste krav vi satte i starten af projektet, herunder at brugeren kan spille spil, få registreret sine highscores og se dem på et leaderboard. Derudover har vi implementeret profiler, så man kan skifte mellem brugere, samt en LAN funktion ved hjælp af

Flask, som gør det muligt at dele highscores over et lokalt netværk. Dette er med til at gøre programmet mere interaktivt og socialt.

Arbejdsprocessen har givet os en bedre forståelse for hvordan man opbygger større programmer og hvordan man får de forskellige lag til at arbejde sammen. Især logik laget har været vigtigt, da det fungerer som bindeled mellem data og brugergrænsefladen. Vi har også fået erfaring med spiludvikling, herunder hvordan man håndterer game states, point og brugerinput.

Brugen af AI har været en hjælp i forhold til at forstå nogle ting som vi ikke havde en ligeså dyb viden om og få forslag til løsninger, især inden for områder som vi ikke havde så meget erfaring med som for eksempel Flask og JSON håndtering. Vi har dog været kritiske overfor de svar vi har fået, og brugt det som et værktøj fremfor at lade det lave arbejdet for os.

Der har været nogle begrænsninger i projektet, blandt andet i forhold til test af GUI da dette er svært at automatisere. Derfor har vi i stedet testet disse dele manuelt. De dele af programmet som vi har kunnet teste automatisk, har fungeret som forventet uden fejl.

Samlet set vurderer vi at projektet er en succes. Programmet opfylder opgaveformuleringen og viser en forståelse for både programmering og program struktur.

Bibliografi

Arrange-Act-Assert: A Pattern for Writing Good Tests. (u.d.). Hentet fra Automation Panda:
<https://automationpanda.com/2020/07/07/arrange-act-assert-a-pattern-for-writing-good-tests/>

Bilag

Figur 1 - menu	5
Figur 2 - trex runner	6
Figur 3 - orb defender	7
Figur 4 - leaderboard	8
Figur 5 - flask	9
Figur 6 - profiles	10
Figur 7 – flowchart	12
Figur 8 - gem til JSON	12
Figur 9 - load scores til Flask	13
Figur 10 - Dictionary eksempel	13